

Python Database Programming

Storage Areas

as the part of our Applications, we required to store our data like Customers Information, Billing Information, Calls Information etc.

To store this data, we require storage areas. There are 2 types of storage areas.

1) Temporary Storage Areas

2) Permanent Storage Areas

1. Temporary Storage Areas:

these are the Memory areas where data will be stored temporarily.

Eg: Python objects like List, Tuple, Dictionary.

Once Python program completes its execution, then these objects will be destroyed automatically and data will be lost.

2. Permanent Storage Areas:

also known as Persistent Storage Areas. here we can store data permanently.

Eg: File Systems, Databases, data warehouses, Big Data technologies etc

File Systems:

File systems can be provided by local operating System. File systems are best suitable to store very less Amount of Information.

Limitations:

- 1) we can not store huge amount of Information.
- 2) there is no Query language support and hence operations will be come very complex.
- 3) there is no security for Data.
- 4) There is no mechanism to prevent duplicate data. Hence, there may be a chance of data inconsistency problems.

to over come the above problems of File Systems, we should go for Databases.

Databases:

- 1) we can store Huge amount of information in the Databases.



- 2) Query Language support is available for every database and hence we can perform Database Operations very easily.
- 3) To access data present in the database, a compulsory username and password are required. Hence, data is secured.
- 4) Inside the database, data will be stored in the form of tables. While developing database table schemas, the database administrator follows various normalization techniques and can implement various constraints like Unique Key Constraints and Primary Key Constraints, which prevent data duplication. Hence, there is no chance of data inconsistency problems.

Limitations of Databases:

- 1) database can not hold very Huge amount of information like Terabytes of Data.
- 2) database can provide support only for Structured Data (Tabular Data OR Relational Data) and can not provide support for Semi Structured Data (like XML Files) and Unstructured Data (like Video Files, Audio Files, Images etc)

to overcome these problems we should go for more Advanced Storage areas like Big Data Technologies, Data warehouses etc.

Python Database Programming:

Sometimes, as part of programming requirements, we have to connect to the database and perform several operations like creating tables, inserting data, updating data, deleting data, and selecting data, etc.

we can use SQL language to talk to the database and we can use Python to send those SQL commands to the database.

Python provides built-in support for several databases like Oracle, My SQL, SQL Server, Gad Fly, SQLite, etc.

Python has separate modules for each database.

**Eg: cx_Oracle module for communicating with Oracle database
pymssql module for communicating with Microsoft SQL Server**

Standard Steps for Python Database Programming:

1. import database specific module

Eg: import cx_Oracle

[Type text]

2. Establish connection between Python program and database.

we can create this connection object by using connect()function of the module. con = cx_Oracle. connect(database information)

Eg: con=cx_Oracle.connect('scott/tiger@localhost')

3. to execute our SQL queries and to hold results some special object is required, which is nothing but Cursor object. We can create Cursor object by using cursor() method.



```
cursor=con.cursor()
```

4. Execute SQL Queries by using cursor object. for this we can use the following methods

- i) `execute(sqlquery)` → To execute a single SQL query
- ii) `executescript(sqlqueries)` → To execute a string of SQL queries separated by semicolon ';'
- iii) `executemany()` → To execute a parameterized query

Eg: `cursor.execute("select * from employees")`

5. commit or rollback changes based on our requirements in the case of DML Queries (insert|update|delete)

commit() Saves the changes to the database
rollback() Rolls all temporary changes back

6. Fetch the result from the cursor object in the case of select queries

- `fetchone()` → To fetch only one row
- `fetchall()` → To fetch all rows and it returns a list of rows
- `fetchmany(n)` → To fetch first n rows

Eg1: `data=cursor.fetchone()`
`print(data)`

Eg2: `data=cursor.fetchall()`
`for row in data:`
`print(row)`

7. close the resources

After completing our operations, it is highly recommended to close the resources in the reverse order of their opening by using `close()` methods.

```
cursor.close()  
()  
con.close()
```

Note: the following is the list of all important methods which can be used for Python database programming.

```
connect()  
cursor()  
execute()  
executescrip
```

```
t()  
executemany(  
    ) commit()  
rollback()  
fetchone()  
fetchall()  
fetchmany(n)
```



`fetch`
`close()`

These methods won't be changed from database to database and are the same for all databases.

Working with Oracle Database:

From Python program, if we want to communicate with any database, some translator must be required to translate Python calls into database-specific calls and database-specific calls into Python calls. This translator is nothing but a Driver/Connector.

Diagram

For Oracle database, the name of the driver needed is `cx_Oracle`.

How to Test Installation:

from Python console execute the following command:

```
>>> help("modules")
```

In the output, we can see `cx_Oracle`.



App1: program to connect with Oracle database and print its version.

```
1) import cx_Oracle
2) con=cx_Oracle.connect('scott/tiger@localhost')
3) print(con.version)
```

Output:

```
D:\python_classes>pydb1
.py 11.2.0.2.0
```

App2: Write a program to create employees table in the Oracle database:

employees(eno, ename, esal, eaddr)

```
1) import cx_Oracle
2) try:
3)     con=cx_Oracle.connect('scott/tiger@localhost')
4)     cursor=con.cursor()
5)     cursor.execute("createtableemployees(enonumber,enamevarchar2(10),esalnumber(10,2),eaddrvarchar2(10))")
6)     print("Tablecreatedsuccessfully")
7) except cx_Oracle.DatabaseError as e:
8)     if con:
9)         print("Thereisaproblemwithsql",e)
10)        con.rollback()
12)    if cursor:
14)        if con:
15)            con.close()
```

App3: Write a program to drop employees table from Oracle database?

```
1) import cx_Oracle
2) try:
3)     con=cx_Oracle.connect('scott/tiger@localhost')
4)     cursor=con.cursor()
5)     cursor.execute("droptableemployees")
6)     print("Tabledroppedsuccessfully")
8)     if con:
7) except cx_Oracle.DatabaseError as e:
10)        print("Thereisaproblemwithsql",e)
12)    if cursor:
9)        con.rollback()
14)    if con:
15)        con.close()
```

App3: Write a program to insert a single row in the employees table.



```
1) import cx_Oracle
2) try:
3)     con=cx_Oracle.connect('scott/tiger@localhost')
4)     cursor=con.cursor()
5)     cursor.execute("insert into employees values(100,'Durga',1000,'Hyd')")
6)     con.commit()

8) except cx_Oracle.DatabaseError as e:
9)     print("Records inserted successfully")
10)     con.rollback()

12) finally:
13)     if con:
14)         cursor.close()
15)         print("There is a problem with sql", e)
16)     con.close()
```

Note: while performing DML Operations(insert|update|delete), compulsory we have to use commit() method, then only the results will be reflected in the database.

App4: Write a program to insert multiple rows in the employees table by using executemany() method.

```
1) import cx_Oracle
2) try:
3)     con=cx_Oracle.connect('scott/tiger@localhost')
4)     cursor=con.cursor()
5)     sql="insert into employees values(:eno,:ename,:esal,:eaddr)"
6)     records=[(200,'Sunny',2000,'Mumbai'),

8)               (400,'Bunny',4000,'Hyd')],
7)             (300,'Chinny',3000,'Hyd')],
10)     con.commit()

12) except cx_Oracle.DatabaseError as e:
13)     cursor.executemany(sql,records)
14)     con.rollback()
15)     print("Records inserted successfully")
16) finally:
18)     if cursor:
19)         cursor.close()
20)     con.close()
```

App5: Write a program to insert multiple rows in the employees table with dynamic input from the keyboard?

```

1) import cx_Oracle
2) try:
3)     con=cx_Oracle.connect('scott/tiger@localhost')
4)     cursor=con.cursor()
5)     while True:
6)         eno=int(input("EnterEmployeeNumber:"))
7)         ename=input("EnterEmployeeName:")
8)         esal=float(input("EnterEmployeeSalary:"))
9)         eaddr=input("EnterEmployeeAddress:")
10)        sql="insert into employees values(%d,'%s',%f,'%s')"

12)        print("Record Inserted Successfully")
11)        cursor.execute(sql%(eno,ename,esal,eaddr))

14)        if option=="No":

16)            break
13)        option=input("Do you want to insert one more record [Yes|No]:")

18)        if con:

20)            print("There is a problem with sql:",e)
19)            con.commit()

22)            if cursor:
17) except cx_Oracle.DatabaseError as e:

24)         if con:
25)             con.close()

```



App6: Write a program to update employee salaries with increment for the certain range with dynamic input.

Eg: Increment all employee salaries by 500 whose salary < 5000

```
1) import cx_Oracle
2) try:
3)     con=cx_Oracle.connect('scott/tiger@localhost')
4)     cursor=con.cursor()
5)     increment=float(input("EnterIncrementSalary:"))
6)     salrange=float(input("EnterSalaryRange:"))

8)     cursor.execute(sql%(increment,salrange))
7)     sql="updateemployees set esal=esal+%f where esal<%f"
10)    con.commit()
9)    print("RecordsUpdatedSuccessfully")
12)    if con:

14)    print("There is a problem with sql:",e)
11) except cx_Oracle.DatabaseError as e:
16)     if cursor:

18)     if con:
19)         con.close()
```

App7: Write a program to delete employees whose salary greater provided salary as dynamic input?

Eg: delete all employees whose salary > 5000

```
1) import cx_Oracle
2) try:
3)     con=cx_Oracle.connect('scott/tiger@localhost')
4)     cursor=con.cursor()
5)     cutoffsalary=float(input("EnterCutOffSalary:"))
6)     sql="delete from employees where esal>%f"
7)     cursor.execute(sql%(cutoffsalary))
8)     print("RecordsDeletedSuccessfully")

10) except cx_Oracle.DatabaseError as e:
9)     con.commit()
12)     con.rollback()

14) finally:
11)     if con:
16)         cursor.close()

18)     con.close()
```





Diagram

We have to download the connector separately from MySQL database.
<https://dev.mysql.com/downloads/connector/python/2.1.html>

How to Check Installation:

from Python console we have to use
`help("modules")`

in the list of modules, compulsory MySQL should be there.

Note: in the case of Python3.4 we have to set PATH and PYTHONPATH explicitly

PATH=C:\Python34
PYTHONPATH=C:\Python34\Lib\site-packages

Q. Write a program to create table, insert data and display data by using MySQL database.

```
1) import mysql.connector
2) try:
3)     con=mysql.connector.connect(host='localhost',database='durgadb',user='root',password='root')
4)     cursor=con.cursor()
5)     cursor.execute("createtableemployees(enoint(5)primarykey,enamevarchar(10),esaldouble(10,2),eaddrvarchar( 10))")
6)     print("TableCreated...")
7)
8)     sql="insertintoemployees(eno,ename,esal,eaddr)VALUES(%s,%s,%s,%s)"
9)
10)    records=[(100,'Sachin',1000,'Ranchi'),
11)             (200,'Kohli',2000,'Delhi')]
12)    cursor.executemany(sql,records)
13)    print("RecordsInsertedSuccessfully..")
14)    cursor.execute("select*fromemployees")
15)    for row in cursor.fetchall():
16)        print("EmployeeName:",row[1])
17)        print("EmployeeAddress:",row[3])
18)        print("EmployeeSalary:",row[2])
19)        print("EmployeeNumber:",row[0])
20)        print("Thereisaproblemwithsql:",e)
21)    if cursor:
22)        print("EmployeeSalary:",row[2])
23)    if con:
```

Q. Write a program to copy data present in employees table of MySQL database into Oracle database.

```
1) import mysql.connector
2) import cx_Oracle
4) con=mysql.connector.connect(host='localhost',database='durgadb',user='root',password='root')
```



```
5) cursor=con.cursor()
6) cursor.execute("select*fromemployees")
7) data=cursor.fetchall()
8) list=[]
9) forrowindata:
10)     t=(row[0],row[1],row[2],row[3])
11)     list.append(t)
12) exceptmysql.connector.DatabaseErroras:
13)     ifcon:
14)         con.rollback()
15)         print("ThereisaproblemwithMySQL:",e)
16) finally:
17)     ifcursor:
18)         cursor.close()
19)     ifcon:
20)         con.close()
21)
22) try:
23)     con=cx_Oracle.connect('scott/tiger@localhost')
24)     cursor=con.cursor()
25)     sql="insertintoemployeesvalues(:eno,:ename,:esal,:eaddr)"
26)     cursor.executemany(sql,list)
27)     con.commit()
28)     print("RecordsCopiedfromMySQLDatabaseetoOracleDatabaseSuccessfully")
29) exceptcx_Oracle.DatabaseErroras:
30)     ifcon:
31)         con.rollback()
32)         print("Thereisaproblemwithsql",e)
33) finally:
34)     ifcursor:
35)         cursor.close()
36)     ifcon:
37)         con.close()
```

[https://dev. MySQL. com/downloads/connector/Python/2. 1. html](https://dev.mysql.com/downloads/connector/Python/2.1.html)

```
1) createtableemployees(enoint(5)primarykey,enamevarchar(10),esaldouble(10,2),eaddrvarchar(10));
2)
3) insertintoemployeesvalues(100,'Durga',1000,'Hyd');
4) insertintoemployeesvalues(200,'Ravi',2000,'Mumbai');
5) insertintoemployeesvalues(300,'Shiva',3000,'Hyd');
```